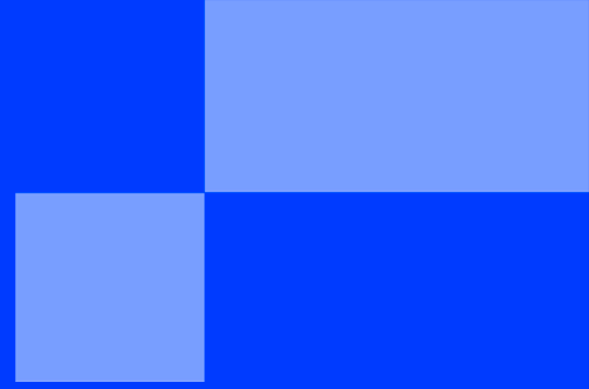




SUMMER SCHOOL

< PART 4 - THE BACK-END ENGINE />

SUMMER SCHOOL

< WEB DEVELOPMENT: BEHIND THE SCENES />

"Data is a precious thing and will last longer than the systems themselves."
— **Tim Berners-Lee**

The Case

The React newspaper looks incredible and is finally interactive. However, the Editor-in-Chief just discovered a fatal flaw: every time he refreshes the page, all the "Likes" disappear, and any new article drafted vanishes into the void.

Currently, your React application has amnesia. It runs locally in the user's browser (the **Front-End**) and forgets everything when closed. To run a real newspaper, you need a **Back-End**: a server running 24/7 that stores the articles safely in a database and sends them to the Front-End when requested.

Your mission today is to build the engine of the newspaper: an **API (Application Programming Interface)** using Node.js and Express. You will not connect it to your React app today. You will build the engine in isolation and test it using a professional developer tool: **Postman**.

Project Requirements

You will build a simple API that allows journalists to publish articles and readers to read them.

1. Feature: The Article Vault (GET)

Your server must be able to serve the list of current articles to anyone who asks.

- Create a file named `articles.json` to act as your database and put some initial dummy articles inside.
- Create a GET `/api/articles` endpoint.
- **Acceptance Criteria:** When calling this endpoint via Postman, the server must read the JSON file and return the array of articles with a `200 OK` status.

Expected Postman Output:

```
[
  {
    "id": 1,
    "title": "Campus Life Returns to Normal",
    "excerpt": "Students are finally getting some rest...",
    "author": "Jane Doe",
    "likes": 5
  }
]
```

2. Feature: The Publishing System (POST)

The journalists need a way to save new stories permanently.

- Create a `POST /api/articles` endpoint.

- **Acceptance Criteria:** The endpoint must receive a JSON object representing a new article. The server must automatically assign it a unique `id`, save it to the `articles.json` file permanently, and return a `201 Created` status with a success message.

Postman Request Body (raw -> JSON):

```
{
  "title": "Epitech Summer School is a massive success",
  "excerpt": "Students build full-stack APIs in record time.",
  "author": "Jane Doe"
}
```

Expected Postman Output:

```
{
  "message": "Article published successfully!",
  "article": {
    "id": 2,
    "title": "Epitech Summer School is a massive success",
    "excerpt": "Students build full-stack APIs in record time.",
    "author": "Jane Doe",
    "likes": 0
  }
}
```

3. Feature : Modify an Article (PATCH)

The Editor-in-Chief wants to be able to modify articles after they are published.

- Create a PATCH `/api/articles/:id` endpoint.

- **Acceptance Criteria:** The endpoint must receive a JSON object with the fields to update. The server must find the article with the specified `id`, update its fields, save the changes to the `articles.json` file, and return a `200 OK` status with a success message. If the article does not exist, return a `404 Not Found` error.

Postman Request Body (raw -> JSON):

```
{
  "title": "Improved Epitech Summer School is a massive success",
  "likes": 10
}
```

Expected Postman Output:

```
{
  "message": "Article updated successfully!",
  "article": {
    "id": 2,
    "title": "Improved Epitech Summer School is a massive success",
    "excerpt": "Students build full-stack APIs in record time.",
    "author": "Jane Doe",
    "likes": 10
  }
}
```

Bonus (Optional)

If your GET, POST, and PATCH routes work perfectly in Postman, try implementing these advanced endpoints to impress the CTO:

- ✓ **Specific Search (GET `/api/articles/:id`):** Create a route that takes an ID directly in the URL. If the user asks for `/api/articles/2`, the server should search the array and return *only* the article with ID 2. If it doesn't exist, return a `404 Not Found` error.
- ✓ **The Eraser (DELETE `/api/articles/:id`):** The Editor-in-Chief wants the ability to delete bad articles. Create a route that completely removes the article with the specified ID from the JSON file and returns a success message.

v1

{EPITECH}